VECTOR

MICROSAR I2C

Technical Reference

DrvI2c_Tc2xxInfi2c01Asr

Version 1.00.00

Authors	Arne van Mourik
Status	Released

Document Information

History

Author	Date	Version	Remarks
Arne van Mourik	2016-04-05	1.00.00	Initial version of this technical reference based on I2c Core.

Reference Documents

No.	Source	Title	Version
[1]	VECTOR	TechnicalReference_I2c_general.pdf	2.01.02

Derivative	Compiler	Document	Hardware Manufacturer Document Version
TC277TF	Tasking_v4.2.2	UM10204_I2c_Spec.pdf	Rev 03
TC277TF	Tasking_v4.2.2	tc27xC_um_v1.5_UsersManual_rev1v5_2013-12.pdf	V1.5

This technical reference describes the specific use of the I2c driver of Aurix Tc27x series.



Caution
We have configured the programs in accordance with your specifications in the questionnaire. Whereas the programs do support other configurations than the one specified in your questionnaire, Vector´s release of the programs delivered to your company is expressly restricted to the configuration you have specified in the questionnaire.

Contents

1 Component History 5

2 Introduction 6

3 Integration 7

3.1 Scope of Delivery 7

3.2 Dependencies on SW modules 8

3.2.1 PORT 8
3.2.2 MCU 9
3.3 Dependencies on HW modules 10

4 Extended API description 11
4.1 Interrupt Service Routines provided by I2C 11
4.1.1 I2C interrupt services 11

5 Configuration 13
5.1 Configuration with DaVinci Configurator Pro 13
5.1.1 General configuration hints 13
5.1.2 I2C bus configuration 14
5.1.3 I2C hardware configuration 15
Base address and interrupts 18

6 Glossary and Abbreviations 19
6.1 Glossary 19
6.2 Abbreviations 19

7 Contact 20

Illustrations

Figure 3-1 PORT example configuration for I2c clock line. 8
Figure 3-2 PORT example configuration for I2c data line. 9
Figure 3-3 Assign the clock for the I2C 10
Figure 5-1 Configuration items in the I2cGeneral container. 13
Figure 5-2 Bus configuration 14
Figure 5-3 Hardware configuration 15
Figure 5-4 Base addresses of the peripheral channels 18

Tables

Table 1-1 Component history 5
Table 3-1 Static files 7
Table 3-2 micro controller compatibility 10
Table 4-1 I2c_IrqUnitBreq 11
Table 4-2 I2c_IrqUnitLbreq 11
Table 4-3 I2c_IrqUnitProt 12
Table 6-1 Glossary 19
Table 6-2 Abbreviations 19

1 Component History

The component history gives an overview over the important milestones that are supported in the different versions of the component.

Component Version	New Features
1.00.00	Initial creation of a simple AUTOSAR like I2C driver.

Table 1-1 Component history

2 Introduction

This document is a supplement to [1] TechnicalReference_Asr_I2c_general.pdf. It describes specific peculiarities of DrvI2c_Tc2xxInfi2c01Asr.

3 Integration

This chapter gives necessary information for the integration of the MICROSAR I2C into an application environment of an ECU.

3.1 Scope of Delivery

The delivery of the I2C contains all static and dynamic file that are listed in [1] chapter 4.1 plus the following file:

File Name	Source Code Delivery	Object Code Delivery	Description
DrvI2c_Tc2xxIfi2c01Asr.jar	■	■	Specific generator file.

Table 3-1 Static files

3.2 Dependencies on SW modules

3.2.1 PORT

Basic Knowledge

The pins SDA and SCL have to be configured in “open drain” mode. Otherwise it is possible that a master receiver will not work. Example for SCL (P02.5) and SDA (P02.4):

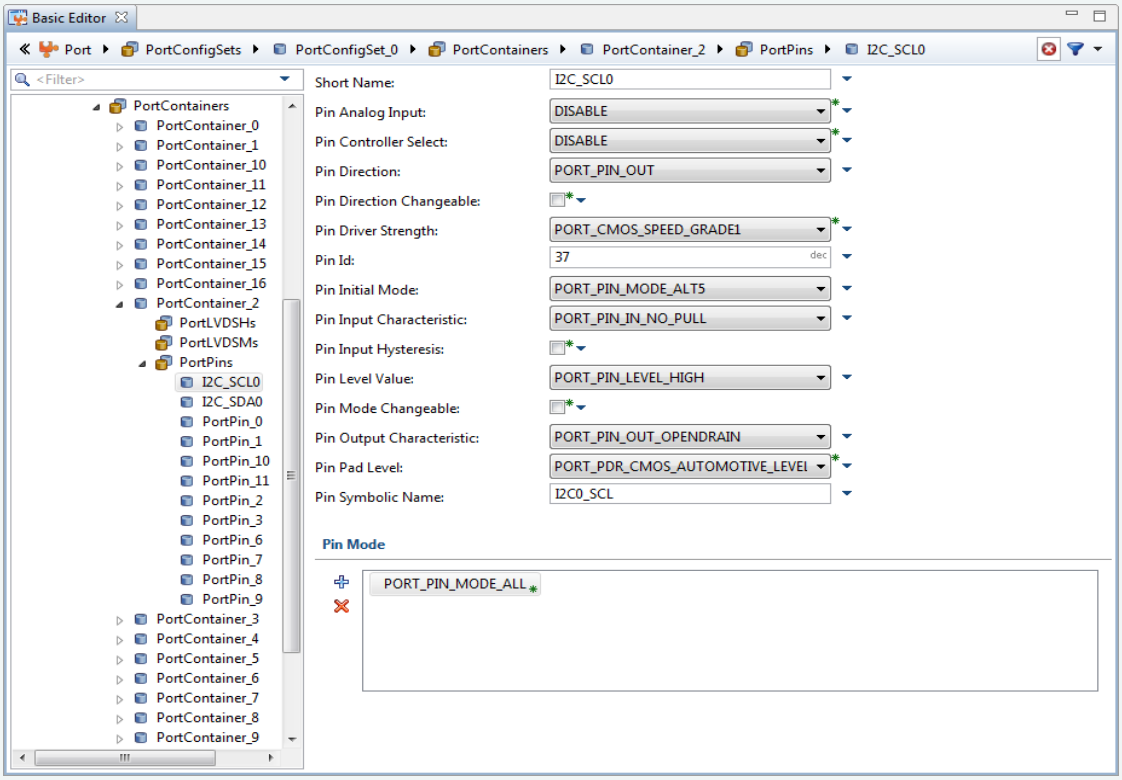


Figure 3-1 PORT example configuration for I2c clock line.

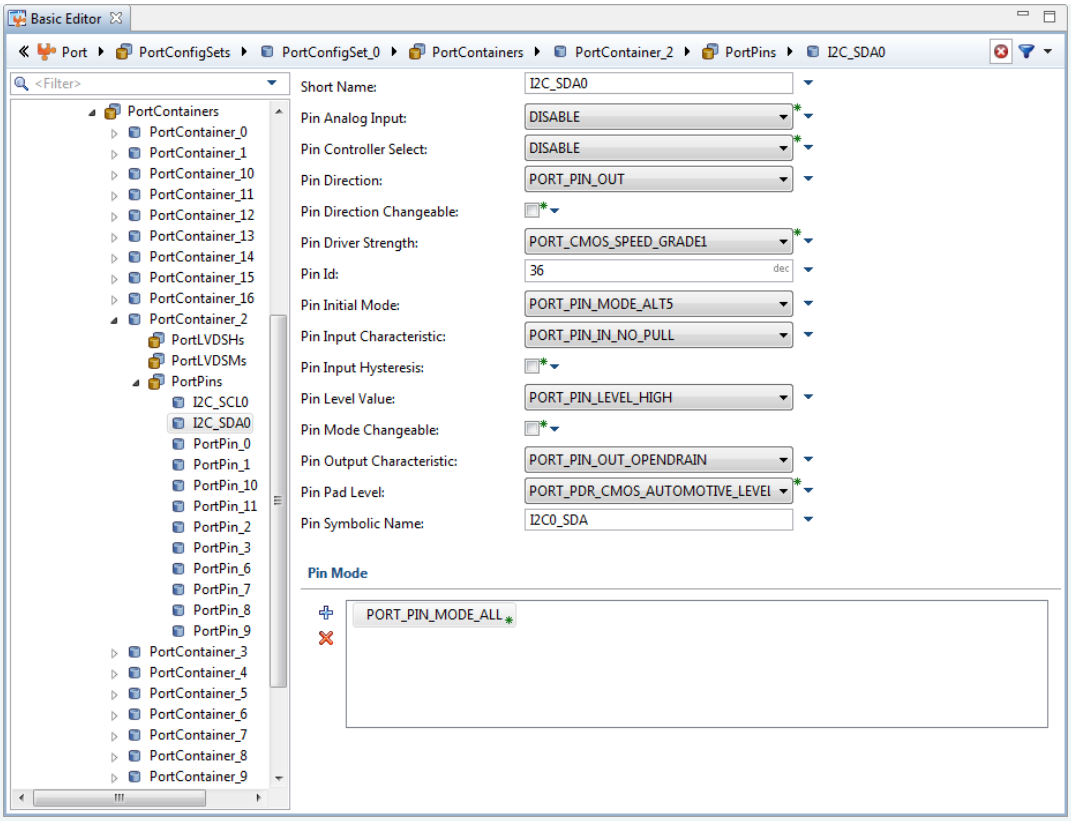


Figure 3-2 PORT example configuration for I2c data line.

3.2.2 MCU

The clock reference point of the I2C module is stored in the parameter “McuClockReferencePoint\McuClockBAUD1Frequency”. Due to the fact that the delivered BSWMD from the chip manufacturer is not 100% AUTOSAR compliant, the clock for the I2C has to be set manually. Get the value from the MCU parameter and put it into the parameter “Set manual clock for I2c in MHz” in the “HardwareUnits” container. The parameter “Select clock source” must be not checked.

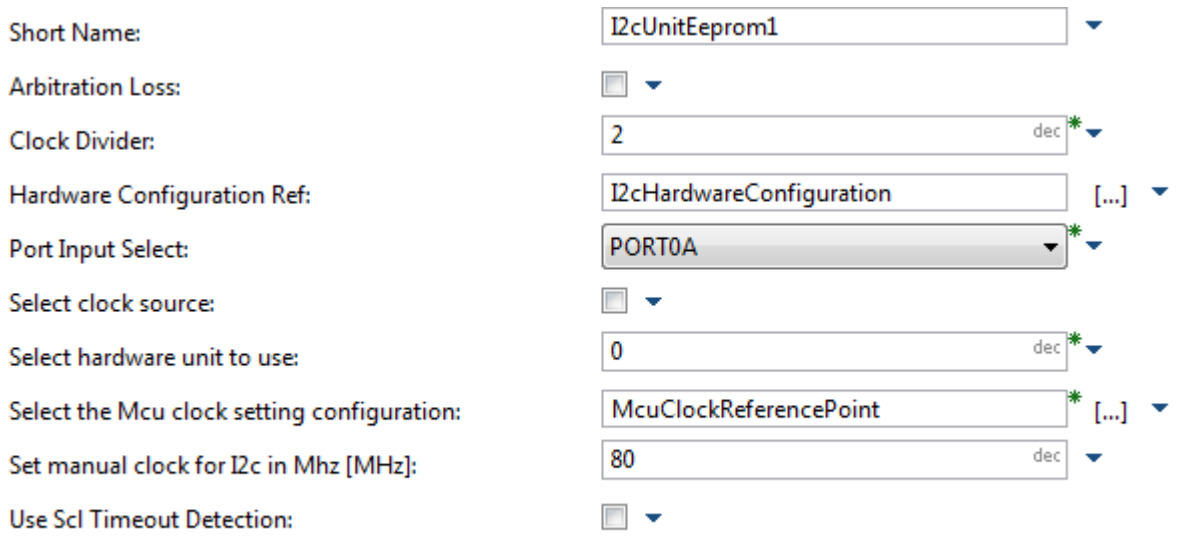


Figure 3-3 Assign the clock for the I2C

3.3 Dependencies on HW modules

The I2C driver is fully compatible (tested) the following derivatives containing an I2C unit:

Serial interface version identifier	Compatibility [full / restricted / no]
Aurix TC277TF	[■ / - / -]
Aurix TC27x Series	[- / ■ / -]

Table 3-2 micro controller compatibility

Note
"restricted" signs that the driver is compatible but has not been tested on this unit.

Caution
If a derivative is used which is not listed; the hardware configuration container allows configuring the I2C hardware with different base addresses. Be sure to put the correct address or the driver will crash!

4 Extended API description

4.1 Interrupt Service Routines provided by I2C

The I2C provides an interrupt service routine for each hardware unit. At the moment only unit I2C0 is supported.

4.1.1 I2C interrupt services

Prototype	
`void` I2c_IrqUnitBreq `( void )`	
Parameter	
--	--
Return code	
--	--
Functional Description	
Interrupt service for burst interrupts. They are used to refill the FIFO during TX and Rx transmissions.	
Particularities and Limitations	
> This function is synchronous. > This function is non-reentrant. > This function is called in interrupt context of the OS.	

Table 4-1 I2c_IrqUnitBreq

Prototype	
`void` I2c_IrqUnitLbreq `( void )`	
Parameter	
--	--
Return code	
--	--
Functional Description	
Interrupt service for last burst interrupts. They are used to refill the FIFO during TX and Rx transmissions when the transfer completes.	
Particularities and Limitations	
> This function is synchronous. > This function is non-reentrant. > This function is called in interrupt context of the OS.	

Table 4-2 I2c_IrqUnitLbreq

Prototype	
`void` I2c_IrqUnitProt `( void )`	
Parameter	
--	--
Return code	
--	--
Functional Description	
Interrupt service for protocol interrupts. They are used to handle the events: > NACK, not acknowledge, > AR, arbitration loss and > TX_END, transmission end.	
Particularities and Limitations	
> This function is synchronous. > This function is non-reentrant. > This function is called in interrupt context of the OS.	

Table 4-3 I2c_IrqUnitProt



Caution
The interrupts for I2C are enabled by an AUTOSAR OS or must be enabled externally in the interrupt module before using this driver.

5 Configuration

For the I2C module the attributes can be configured according to/ with the following methods/ tools:

- Configuration in DaVinci Configurator (Base/Pro). For a detailed description of the configuration features please refer to the “Properties->Description” window.

5.1 Configuration with DaVinci Configurator Pro

5.1.1 General configuration hints

The driver uses a macro from the board driver package to get access to the protected registers. The macro is preset and can be changed but a wrong mapping leads to an invalid register access and causes a CPU trap.



Basic Knowledge

The macro which is used by the I2C module can be configured. Its mapping is:

```
#define I2C_ENABLE_SAFE_REG_ACCESS     Mcal_ResetENDINIT
#define I2C_DISABLE_SAFE_REG_ACCESS    Mcal_SetENDINIT
```

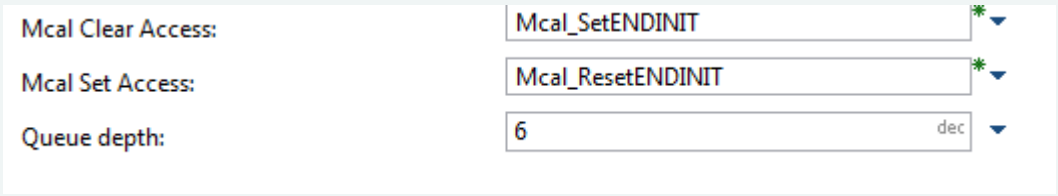


Figure 5-1 Configuration items in the I2cGeneral container.

5.1.2 I2C bus configuration

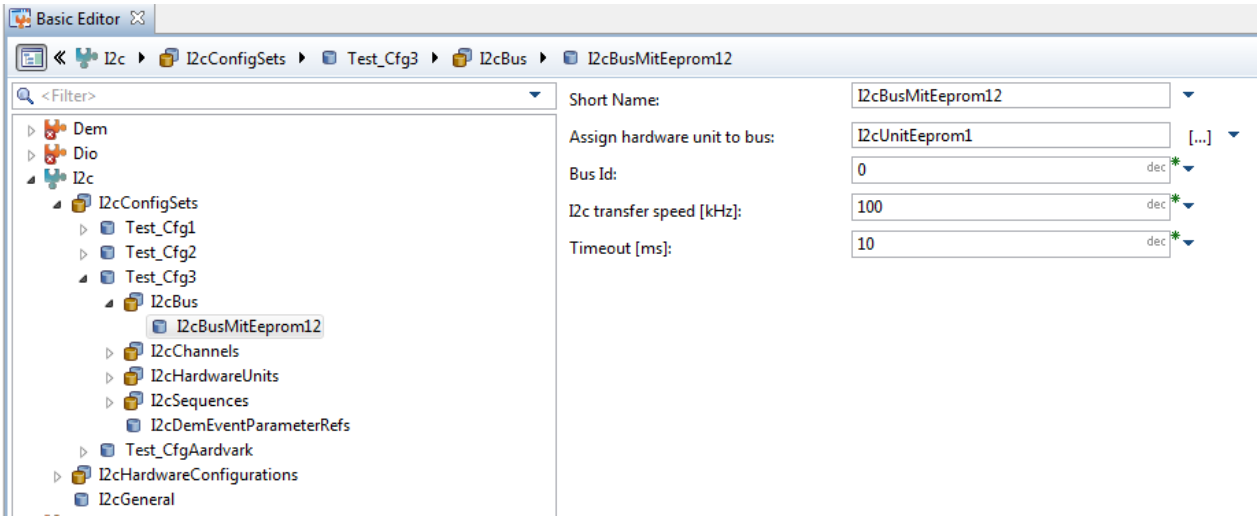


Figure 5-2 Bus configuration

A bus configuration is assigned to a sequence. All channels in a sequence inherit the properties of this bus configuration.

FAQ

How to change the transfer speed on the same hardware unit?

Practical Procedure

A sequence (Sequence_100) is used with three channels and a bus configuration which uses a hardware unit (I2cComPeriChannel0) for transfer. The transfer speed is 100KHz.

- Now create a new sequence (Sequence_400) and a new bus configuration.
- Set the transfer speed in the bus configuration to 400 KHz.
- Assign the same hardware unit (I2cUnitEeprom1) to this bus configuration.
- Assign all channels from Sequence_100 to Sequence_400.

Transmitting Sequence_100 transfers all channels with a baud rate of 100 KHz and Sequence_400 does the same but with 400 KHz.

5.1.3 I2C hardware configuration

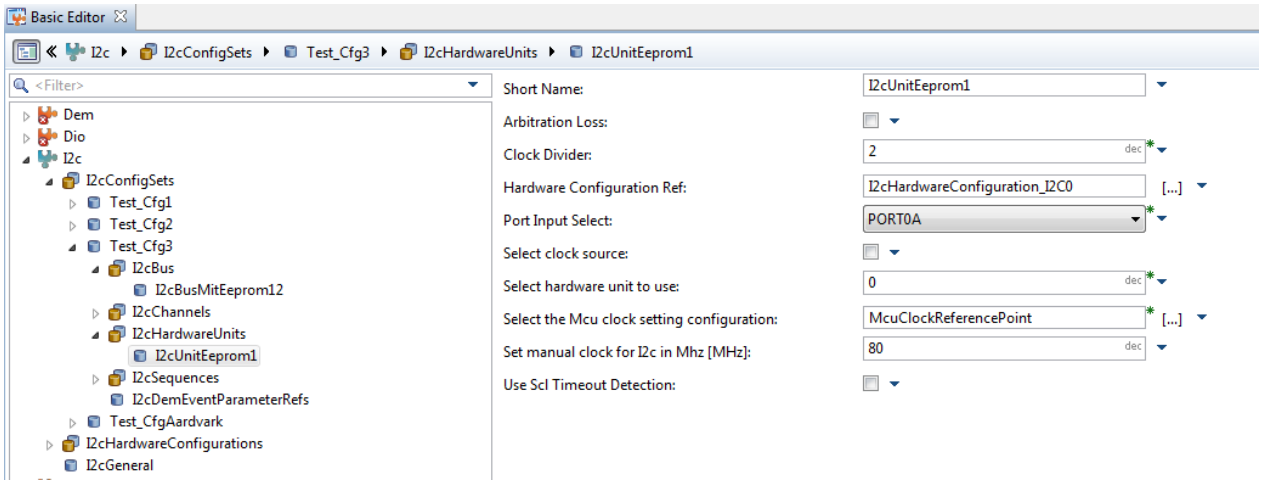


Figure 5-3 Hardware configuration

The pre-configured hardware configurations can be used to assign the base address. If a base address is possible, but not listed in the hardware configurations, a new hardware configuration can be added and selected.

For the “Port input select” parameter please consult the hardware manual (see also example).



Example

For the PORT pins a) P02.4 und b) P15.5 taken from the manual. For correct functionality the port a) must be set to SDA0A and b) to SDA0C to ensure the routing functionality.

Table 14-14 Port 02 Functions (cont'd)

Port Pin	I/O	Pin Functionality	Associated Reg./ I/O Line	Port I/O Control Select.	
				Reg./Bit Field	Value
P02.4	I	General-purpose input	P02_IN.P4	P02_IOCR4. PC4	$0XXXX_B$
		GTM input	TIN4		
		QSPI3 input	SLSI3A		
		CAN node 0 input	RXDCAN0D		
		CCU60 input	CC62INA		
		CCU61 input	CC62INB		
		DSADC input	DSDIN5B		
		I2C0 input	SDA0A		
		CIF input	CIFD4		
	O	General-purpose output	P02_OUT.P4		$1X000_B$
		GTM output	TOUT4		$1X001_B$
		ASCLIN2	ASCLK2		$1X010_B$
		QSPI3 output	SLSO30		$1X011_B$
		PSI5-S output	PSISCLK		$1X100_B$
		I2C0 output	SDA0		$1X101_B$
		ERAY output	TXENA		$1X110_B$
		CCU60 output	CC62		$1X111_B$

Table 14-26 Port 15 Functions (cont'd)

Port Pin	I/O	Pin Functionality	Associated Reg./ I/O Line	Port I/O Control Select.	
				Reg./Bit Field	Value
P15.5	I	General-purpose input	P15_IN.P5	P15_IOCR4. PC5	$0XXXX_B$
		GTM input	TIN76		
		ASCLIN1 input	ARX1B		
		QSPI2 input	MTSR2A		
		I2C0 input	SDA0C		
		SCU input	REQ13		
	O	General-purpose output	P15_OUT.P5		$1X000_B$
		GTM output	TOUT76		$1X001_B$
		ASCLIN1 output	ATX1		$1X010_B$
		QSPI2 output	MTSR2		$1X011_B$
		MSC0 output	END02		$1X100_B$
		MSC0	EN00		$1X101_B$
		I2C0 output	SDA0		$1X110_B$
		CCU60 output	CC61		$1X111_B$

Base address and interrupts

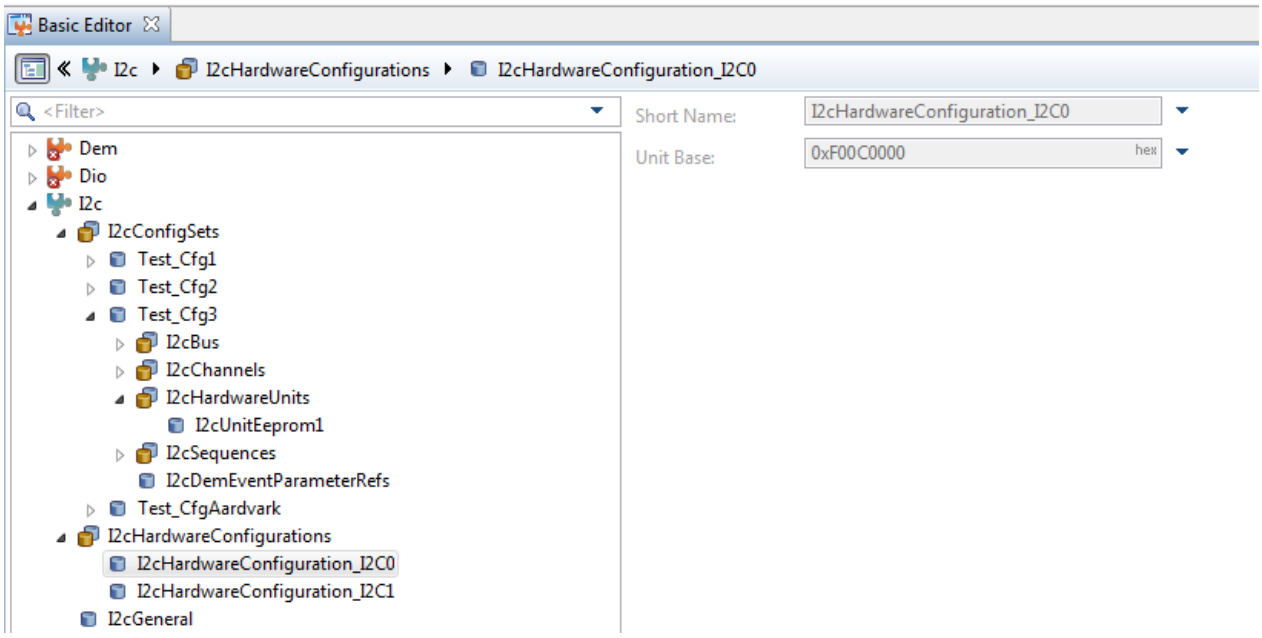


Figure 5-4 Base addresses of the peripheral channels

Caution

Be sure to configure a valid base address if adding a hardware configuration. A misconfiguration of this parameter leads to a CPU crash or can cause an undefined behavior. Also verify the selected hardware configuration fits to the chip derivative.

6 Glossary and Abbreviations

6.1 Glossary

Term	Description
DaVinci Configurator Pro	Configuration and generation tool for MICROSAR components
PORT	A Port represents a whole configurable port on this microcontroller (typically controlled by one hardware register set).
I2C	Inter Integrated Circuit, also known as IIC or I^2C
SDA	Serial Data, data line of the I2c bus.
SCL	Serial Clock, clock line of the I2c bus.

Table 6-1 Glossary

6.2 Abbreviations

Abbreviation	Description
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
ECU	Electronic Control Unit
ISR	Interrupt Service Routine
MICROSAR	Microcontroller Open System Architecture (the Vector AUTOSAR solution)

Table 6-2 Abbreviations

7 Contact

Visit our website for more information on

- News
- Products
- Demo software
- Support
- Training data
- Addresses

www.vector.com